

UNIVERSITÀ DEGLI STUDI DI PISA

Project

PEER TO PEER SYSTEMS
AND BLOCKCHAINS

Decentralised Lending Service

Student

Simone Infantino

123456

Student

Antonio Tamborrino

123456

A.A. 2025/2026

Contents

1	Intro and Implementation Choices	1
1.1	Smart Contract Implementation	1
1.2	Lending Service Architecture	1
1.3	Loan Proposal and Voting Mechanism	2
1.4	Loan Contract Design	2
1.5	Bitcoin Oracle Integration	3
1.6	Security and Administrative Choices	3
2	Reentrancy and Security Discussion	4
2.1	Gas Evaluation	4
2.2	Reentrancy Protection	5
2.3	Discussion of Malicious Strategies	5
2.3.1	Reentrancy Attack	6
2.3.2	Unauthorized Access	6
2.3.3	Invalid Input Manipulation	6
2.3.4	Manipulation of Loan Voting	6
2.3.5	Denial of Service and Gas Limitations	6
3	User Manual	8
3.1	Project Structure	8
3.2	Orchestration Script	8

Chapter 1

Intro and Implementation Choices

1.1 Smart Contract Implementation

The decentralized lending service is implemented through a set of Solidity smart contracts developed and tested using the Hardhat framework. The main goal of the implementation is to provide a lending platform where contributors can supply funds, applicants can request loans backed by Bitcoin collateral information, and the community can participate in the approval process through a voting mechanism.

The smart contract architecture is composed of three main components: the `LendingService` contract, the `Loan` contract, and the `BitcoinOracle` contract. The separation of responsibilities between these contracts improves modularity and allows each component to manage a specific part of the protocol logic.

1.2 Lending Service Architecture

The `LendingService` contract represents the core of the decentralized lending protocol. It manages the contributor liquidity pool, loan proposals, voting mechanism, compensation system, and the interaction with the oracle.

A main design choice was to separate the global lending logic from individual loans. Instead of storing all loan information inside a single contract, each approved proposal generates a new independent `Loan` contract. This approach allows every loan to maintain its own state, including the applicant, principal amount, repayment status, expiration block, and contributors involved.

The contract maintains the amount deposited by each contributor through the `deposited` mapping and tracks the funds currently allocated to active loans through the `locked` mapping. This distinction allows contributors to withdraw only the funds that are not committed to ongoing loans.

The deposit mechanism requires a minimum contribution defined by the `MIN_DEPOSIT` constant. Contributors are stored inside an array to allow iteration during operations such as liquidity evaluation and vote weighting. When a contributor no longer has available funds or locked positions, the address is removed from the contributor list to reduce unnecessary storage usage.

1.3 Loan Proposal and Voting Mechanism

Loan creation follows a proposal-based approach. Applicants submit a proposal containing the requested amount, interest rate, duration, and Bitcoin address used as collateral reference. Each proposal remains open for a fixed number of blocks, defined by the `PROPOSAL_VOTING_PERIOD` constant.

Only contributors with available deposited funds can participate in the voting process. Votes are weighted according to the contributor's disposable liquidity rather than using a simple one-address-one-vote mechanism. This choice prevents contributors with very small deposits from having the same influence as contributors providing a larger amount of liquidity.

When the voting period expires, the proposal is evaluated through multiple conditions:

- the lending pool must contain enough disposable liquidity;
- the Bitcoin collateral value retrieved from the oracle must satisfy the liquidity requirement;
- the weighted approval votes must exceed the rejection weight.

If all conditions are satisfied, the service creates a new Loan contract and locks the required funds proportionally among the contributors.

1.4 Loan Contract Design

The Loan contract represents a single lending agreement. It is deployed dynamically by the `LendingService` contract and receives the collected principal amount during construction. After deployment, the principal is immediately transferred to the applicant.

The contract stores immutable information about the loan, including:

- applicant address;
- principal amount;
- interest rate;
- loan duration;
- collateral percentage;
- Bitcoin address associated with the collateral.

Using immutable variables for these parameters prevents modifications after deployment and reduces unnecessary storage operations.

The repayment mechanism allows partial or complete repayments. Each payment is divided proportionally into principal repayment and interest according to the

configured interest rate. Principal repayments are distributed back to contributors, while interest payments are divided between contributor gains and the compensation pool.

The contributors participating in a loan are stored in a sorted order based on the initially locked amount. This ordering is used during repayment distribution, allowing refunds to be processed according to the initial contribution size.

1.5 Bitcoin Oracle Integration

The lending protocol requires external Bitcoin information to evaluate whether a loan proposal has sufficient collateral. Since Ethereum smart contracts cannot directly access external blockchain data, a dedicated `BitcoinOracle` contract was implemented. The oracle stores Bitcoin balances associated with Bitcoin addresses represented as byte arrays. Applicants can request an update by paying a minimum fee, while the oracle owner is responsible for pushing updated balances after performing the off-chain scan.

The separation between the request and update phases allows the blockchain component to remain independent from the off-chain Bitcoin data acquisition process. The `deploymentBlock` variable is also stored to support the off-chain service in identifying the starting point for synchronization.

1.6 Security and Administrative Choices

The contracts include several mechanisms to improve security and maintainability. The lending service implements an administrative mechanism based on a two-step ownership transfer. Instead of immediately changing the administrator, the current administrator nominates a pending administrator who must explicitly accept the role. This prevents accidental transfers to incorrect addresses.

A similar ownership transfer mechanism is implemented in the oracle contract.

The lending service also includes a termination mechanism that requires a successor contract address and prevents termination while active loans exist. This allows a future migration to a new contract version without leaving unresolved loans.

The implementation also includes callback functions between the `Loan` and `LendingService` contracts. These callbacks are used to notify the service about successful repayments, contributor refunds, interest collateral, and loan outcomes while keeping the two contracts logically separated.

Finally, the contract suite includes a vulnerable version of the lending service and a dedicated reentrancy attacker contract used during testing. These contracts are used to evaluate the impact of reentrancy attacks and validate the effectiveness of the adopted security modifications.

Chapter 2

Reentrancy and Security Discussion

2.1 Gas Evaluation

Gas consumption was evaluated through the Hardhat testing environment, which provides the gas usage of each executed transaction. The implementation choices were analyzed considering the main operations of the lending protocol, such as deposits, proposal resolution, loan creation, repayments, and compensation claims. The contracts were designed to limit unnecessary storage operations, since writing to storage represents one of the most expensive operations in terms of gas consumption. For example, the `Loan` contract uses `immutable` variables for values that are fixed after deployment, such as the applicant address, principal amount, interest rate, and loan duration. This reduces storage usage and improves execution efficiency.

The modular architecture also affects gas consumption. Each approved proposal creates a new `Loan` contract instead of storing all loans inside the main `LendingService` contract. Although contract creation introduces an additional deployment cost, this design improves separation of responsibilities and reduces the complexity of operations performed by the main contract.

Some operations require iteration over dynamic arrays. In particular, `_cumulativeDisposable()`, `_approveWeight()`, and `_lockProportional()` iterate over the contributor list to calculate available liquidity, voting weight, and fund allocation. Moreover, contributors are sorted during loan creation using a selection sort algorithm. These operations increase gas usage proportionally to the number of contributors, but they were considered acceptable for the scope of the project.

The oracle interaction was also designed with gas efficiency in mind. The `BitcoinOracle` contract stores Bitcoin balances on-chain only after an off-chain update performed by the oracle owner. This avoids expensive on-chain Bitcoin blockchain analysis and keeps the Ethereum contract responsible only for verification and storage of the required information.

2.2 Reentrancy Protection

The project includes two versions of the lending service: `LendingService.sol`, containing the protected implementation, and `LendingServiceVulnerable.sol`, intentionally introducing a reentrancy vulnerability for testing purposes.

The vulnerability is present in the `claimCompensation()` function. In the vulnerable implementation, the contract performs an external Ether transfer before updating its internal state:

1. The compensation amount is calculated.
2. Ether is transferred to the contributor.
3. Internal accounting variables are updated.

Since the recipient can execute arbitrary code during the external call, a malicious contract can re-enter the `claimCompensation()` function before the previous execution has completed. The attacker can therefore request multiple compensations using the same accounting state.

The attack scenario is implemented in the `ReentrancyAttacker.sol` contract. The attacker contract receives Ether through the fallback function and recursively calls `claimCompensation()` while the vulnerable contract has not yet updated the contributor balances.

The secure implementation applies the Checks–Effects–Interactions pattern by performing state updates before external calls:

1. Check the validity of the request and the loan status.
2. Update the compensation pool and contributor accounting variables.
3. Perform the final Ether transfer.

By changing the execution order, the attacker cannot reuse the same compensation position during a recursive call because the contract state already reflects the previous operation.

The effectiveness of this modification is verified through the dedicated Hardhat tests: `Reentrancy.test.ts` evaluates the attack against the vulnerable contract, while `ReentrancySafe.test.ts` verifies that the protected implementation prevents the same malicious behaviour.

2.3 Discussion of Malicious Strategies

Several malicious strategies were considered during the implementation and testing phase.

2.3.1 Reentrancy Attack

The main security issue analyzed is the reentrancy vulnerability in the compensation mechanism. The vulnerable implementation allows an attacker contract to recursively call external functions before the contract state is updated. The attack is mitigated in the secure version by applying the Checks–Effects–Interactions pattern.

2.3.2 Unauthorized Access

Administrative operations are protected through the `onlyAdmin` modifier. Functions such as oracle replacement, service termination, and administrator transfer can only be executed by the current administrator.

The ownership transfer mechanism uses a two-step process. The current administrator first nominates a pending administrator, who must explicitly accept the role. This prevents accidental transfers to invalid addresses.

2.3.3 Invalid Input Manipulation

All externally callable functions validate input parameters using Solidity `require` statements. Examples include:

- minimum deposit requirements;
- valid interest rate ranges;
- positive loan amounts and durations;
- valid proposal states;
- valid oracle and administrator addresses.

These checks prevent inconsistent states from being introduced through malformed transactions.

2.3.4 Manipulation of Loan Voting

The voting system does not use a simple one-address-one-vote mechanism. Instead, voting power is weighted according to the contributor’s disposable liquidity. This prevents accounts with negligible deposits from having the same influence as major liquidity providers.

2.3.5 Denial of Service and Gas Limitations

The implementation relies on iterations over contributor lists for some operations. Although necessary for proportional fund allocation and weighted voting, these loops could become expensive if the number of contributors grows significantly.

For this reason, dynamic operations are limited to the contributor set maintained by the contract, and contributors with no remaining deposited or locked funds are removed from the list.

Overall, the final implementation follows Solidity security best practices and was validated by comparing the vulnerable and secure implementations through dedicated security tests.

Chapter 3

User Manual

3.1 Project Structure

The project is organized into several modules, each responsible for a specific component of the decentralized lending platform. The directory structure is designed to clearly separate blockchain data, smart contracts, off-chain services, deployment scripts, and utility tools.

- **bt_chain_blocks/**: local copy of the Bitcoin blockchain blocks used by the off-chain scanner.
- **chain/**: contains the local Ethereum blockchain configuration, including the genesis file, account password, and Geth data directory.
- **hardhat/**: contains the Solidity smart contracts, deployment scripts, and Hardhat configuration used during development and testing.
- **offchain-scanner/**: Java application that scans the Bitcoin blockchain and extracts UTXO information required by the lending protocol.
- **python/**: includes all Python scripts responsible for contract deployment, oracle management, automated voting, and the demonstration workflow.
- **state/**: stores serialized state files generated during the deployment process, allowing the various Python components to share deployment information.
- **tool.py**: orchestration script that provides a unified interface for setting up and running the entire system.
- **venv/**: Python virtual environment containing all required dependencies.

3.2 Orchestration Script

The entire project can be managed through the `tool.py` Python script, which acts as an orchestration tool. It provides an interactive command-line menu that executes the required Linux shell commands and external programs (such as Geth, Maven, and the Python utilities) to set up and run the complete system. This approach simplifies the

deployment process by automating the execution of individual terminal commands behind a single interface.

The available operations are summarized below.

1. **Copy Bitcoin blocks and run the Off-chain Scanner.** This operation copies the local Bitcoin blockchain into the project directory, compiles the Java scanner using Maven, and generates the UTXO snapshot required by the protocol. This step only needs to be executed once.
2. **Setup the Python virtual environment.** A Python virtual environment is created (if not already available) and the required dependencies are installed automatically.
3. **Initialize the local Ethereum blockchain.** Existing Geth chain data are removed and a new blockchain is initialized from the provided genesis configuration.
4. **Start the local Geth node.** Launches the private Ethereum blockchain with mining enabled, unlocks the predefined account, and exposes the HTTP JSON-RPC interface used by the application.
5. **Deploy the application.** Executes the deployment script, creates the required accounts, deploys all smart contracts, initializes the protocol state, and stores deployment information inside the `state/` directory.
6. **Start the Oracle Daemon.** Launches the off-chain oracle responsible for monitoring blockchain events and providing external information required by the protocol.
7. **Start the Auto Voter.** Runs the automated voting component that simulates validator participation in governance decisions.
8. **Run the demonstration.** Executes a complete lending workflow demonstrating the interaction among borrowers, lenders, smart contracts, and off-chain services.
9. **Attach to Geth.** Opens the JavaScript console connected to the running Ethereum node, allowing direct inspection of accounts, transactions, and contract state.

To correctly execute the project, the following workflow is recommended. For each step, the user should open a terminal, execute `python3 tool.py`, and select the corresponding menu option. Operations that start long-running services (such as the Geth node, the Oracle Daemon, and the Auto Voter) keep the process active and therefore require the terminal to remain open while the rest of the system is running. Consequently, a new terminal should be opened for each subsequent service.